

# NoteHub要件定義書

## 概要

NoteHubは複数ユーザーが同一ドキュメントを同時に開き、リアルタイムで共同作業で編集できるオンラインエディタ

ユーザーはアカウントの登録、ログイン後にワークスペースを作成し、その中でドキュメントを作成、編集、削除できる

## 目的

チーム単位での作業をリアルタイムで可視化しながら、共同作業できる環境を提供する

## ユーザーへの解決したい課題と内容

| 課題                | 内容                                    |
|-------------------|---------------------------------------|
| 編集内容が同期されず二重管理になる | 最新更新の連携にタイムラグがあり、一つのタスクを二重で実施してしまうリスク |
| 自動保存              | 自動保存                                  |
| 過去の状態に戻せない        | 誤削除、変更した内容を復元する手段がなく、やり直しになるリスク       |

## 実装する機能

| 機能          | 内容                                  |
|-------------|-------------------------------------|
| ユーザー登録・ログイン | Email/Passwordによるアカウント登録、ログイン、ログアウト |
| ワークスペース管理   | ワークスペースの作成・編集・削除、メンバーの招待・削除         |
| ドキュメント管理    | ドキュメントの作成・編集・削除、検索                  |
| リアルタイム同時編集  | WebSocketを利用し複数ユーザー間での編集内容のリアルタイム編集 |
| テキスト編集      | Monaco Editorによるコード・文章の編集           |

|              |                                    |
|--------------|------------------------------------|
| 編集者表示        | 現在ドキュメントを開いている・編集しているユーザーをアイコン等で表示 |
| 自動保存         | 一定間隔・一定操作ごとにドキュメント内容を自動的に保存        |
| 編集履歴・バージョン管理 | 過去のバージョンの一覧表示、任意バージョンへの復元          |

## 技術スタック

| 分類      | 技術スタック                                                                            |
|---------|-----------------------------------------------------------------------------------|
| フロントエンド | React、TypeScript、Vite、Tailwind CSS、React Router、Axios、Monaco Editor、WebSocket API |
| バックエンド  | Go、Echo、GORM、PostgreSQL、Redis、Gorilla WebSocket、bcrypt                            |
| 認証      | JWT                                                                               |
| インフラ    | Docker、Docker Compose                                                             |
| テスト     | Go testing、Vitest                                                                 |
| CI/CD   | GitHub Actions                                                                    |
| API     | REST API、WebSocket                                                                |

## User

| できること           | 内容                                |
|-----------------|-----------------------------------|
| ワークスペースの作成      | 自分をホストのワークスペースを作成できる              |
| ワークスペースへの参加     | ホストから招待されたワークスペースに参加できる           |
| ドキュメントの作成・編集・削除 | 参加しているワークスペース内のドキュメントに対して操作できる    |
| 共同編集            | 同じドキュメントを開いた他メンバーとリアルタイムに編集を共有できる |

## Host

| できること | 内容 |
|-------|----|
|-------|----|

|           |               |
|-----------|---------------|
| メンバー管理    | メンバーの招待・削除    |
| ワークスペース管理 | ワークスペースの作成・削除 |

## リアルタイム共同編集

| 項目     | 内容                                                     | 補足                              |
|--------|--------------------------------------------------------|---------------------------------|
| 通信方式   | Gorilla WebSocket                                      | クライアント⇄サーバー間                    |
| 変更内容反映 | WebSocketサーバーは編集内容をRedisに送ってほかのサーバー（メンバー）にも知らせる        | ント単位のチャンネル配信複数サーバーインスタンス間でも同期可能 |
| 同期単位   | ドキュメント単位                                               | document_id) 同一ドキュメントを開いたユーザーのみ |
| 再接続    | ネットワークが切れたりサーバーとの接続が切断された場合でも、一定間隔で再接続を試行し編集内容を正しく復元する | 再接続時は最新の保存内容を再取得し反映             |

## 自動保存

| 項目      | 内容                                            |
|---------|-----------------------------------------------|
| 保存タイミング | 編集操作から一定時間経過後（60秒）                            |
| 保存先     | PostgreSQL                                    |
| 保存者     | ブラウザではなく、サーバーが保存処理を行う                         |
| 保存方法    | Redis上の最新内容を一定間隔でPostgreSQLへ保存する（Redis単体サーバー） |
| 失敗時     | 保存失敗時はリトライし、クライアントへ保存ステータスを通知する               |
| 保持      | 最新の状態のみ                                       |

## 編集履歴・バージョン管理

| 項目 | 内容 |
|----|----|
|----|----|

|             |                                      |
|-------------|--------------------------------------|
| 復元用の保存タイミング | 自動保存とは別に一定期間経過ごとにバージョンとして保存（10分ぐらい？） |
| 保存内容        | ドキュメント本文、更新者、更新日時                    |
| 復元          | 任意のバージョンを選択し、現在のドキュメントへ復元できる         |
| 保持          | 過去の状態を履歴として残す                        |

## 認証・セキュリティ

| 項目          | 内容                                                  |
|-------------|-----------------------------------------------------|
| 登録          | Email／Password                                      |
| AccessToken | JWT 有効期限を短く設定 メモリ上（Reactの状態管理や変数）だけで保持（ページリロードで消える） |
| パスワード       | ハッシュ化（bcrypt）して保存し、平文は保持しない                         |
| WebSocket認証 | 本当にログイン済みのユーザーか確認（WebSocket開始前に）                    |
| 操作権限        | ワークスペース・ドキュメントへのアクセスは、所属メンバーかどうかをAPI・WebSocketで確認   |

## ユーザー登録

| 項目      | 内容                       |
|---------|--------------------------|
| 入力項目    | Email、表示名、パスワード          |
| Email   | サービス内で一意 重複させない          |
| Email形式 | メールアドレス形式であることを確認する      |
| パスワード要件 | 8～15文字、英字と数字を含む 最大文字数を設定 |
| パスワード保存 | bcryptでハッシュ化して保存する       |
| 平文パスワード | データベース、ログ、レスポンスへ出力しない    |

## ログイン処理内容

| 項目     | 内容                                     |
|--------|----------------------------------------|
| 認証方法   | Email・パスワード認証                          |
| ユーザー確認 | Emailに一致するユーザーを取得し、削除済み・無効化されていないことを確認 |

|         |                                        |
|---------|----------------------------------------|
| パスワード照合 | bcryptでハッシュ化されたパスワードを照合する              |
| 認証成功    | AccessTokenを発行する                       |
| 認証失敗    | Emailまたはパスワードのどちらが誤っているか特定できないメッセージを返す |
| ログ記録    | ログイン成功・失敗をログへ記録する                      |

## データモデル

| テーブル            | 内容                     | 項目                                                                               |
|-----------------|------------------------|----------------------------------------------------------------------------------|
| User            | ユーザーアカウント情報            | id, email, password_hash, name, created_at, updated_at                           |
| Workspace       | ワークスペース情報              | id, name, host_id, created_at, updated_at                                        |
| WorkspaceMember | ワークスペースとメンバーの所属関係・権限管理 | id, workspace_id, user_id, role (Host/Member), joined_at                         |
| Document        | ドキュメント情報               | id, workspace_id, title, content, created_by, updated_by, created_at, updated_at |
| DocumentVersion | ドキュメントの編集履歴            | id, document_id, content, created_by, created_at                                 |
| EditSession     | リアルタイム編集集中のユーザー接続状態    | id, document_id, user_id, connection_id, connected_at, last_active_at            |

## API

| API                           | 内容                                                        |
|-------------------------------|-----------------------------------------------------------|
| POST /auth/signup             | ユーザー登録                                                    |
| POST /auth/login              | ログイン (JWT発行)                                              |
| POST /auth/logout             | ログアウト (AccessToken、RefreshTokenを無効化Cookieに保存されている認証情報を削除) |
| GET /workspaces               | 所属ワークスペース一覧取得                                             |
| POST /workspaces              | ワークスペース作成                                                 |
| POST /workspaces/:id/members  | メンバー招待                                                    |
| GET /workspaces/:id/documents | ドキュメント一覧取得                                                |

|                                |                        |
|--------------------------------|------------------------|
| POST /workspaces/:id/documents | ドキュメント作成               |
| GET /documents/:id             | ドキュメント詳細取得             |
| GET /documents/:id             | GET /documents/:id     |
| GET /documents/:id/versions    | 編集履歴一覧取得               |
| WS /ws/documents/:id           | リアルタイム共同編集用WebSocket接続 |

## AccessToken

| 項目          | 内容                                                |
|-------------|---------------------------------------------------|
| 形式          | JWT                                               |
| 有効期限        | 15分                                               |
| 保持場所        | クライアントのメモリ上のみ（localStorage／sessionStorageには保存しない） |
| 格納情報        | user_id、ワークスペース権限情報、発行日時、有効期限                     |
| 用途          | id, document_id, content, created_by, created_at  |
| EditSession | REST API・WebSocket接続の双方で本人確認に利用                   |

## セキュリティ・運用要件

### タブ開きすぎ防止

| 項目   | 内容                        |
|------|---------------------------|
| 制限対象 | 同一ユーザー・同一ドキュメントの同時接続（タブ）数 |
| 上限   | 同一ドキュメントにつき最大3タブ          |
| 超過時  | 新規接続を拒否                   |

### 同時ログイン端末数制限

| 項目   | 内容                            |
|------|-------------------------------|
| 管理単位 | 1つのRefreshTokenを1つのログインセッション  |
| 上限値  | 1アカウントあたりの同時ログイン可能な数を設定(相談する) |

| 項目      | 内容                                      |
|---------|-----------------------------------------|
| 超過時     | 最も古いセッションを無効化                           |
| セッション管理 | ユーザーは有効なログインセッション一覧を確認し、任意のセッションを無効化できる |

## ログイン試行制限

| 項目    | 内容                    |
|-------|-----------------------|
| 目的    | ブルートフォース攻撃対策          |
| 制限方式  | 一定時間内の連続ログイン失敗回数を管理   |
| 上限値   | 5回失敗で一時ロック            |
| ロック内容 | 15分間ログインを拒否           |
| 記録単位  | アカウント単位およびIPアドレス単位で管理 |
| 通知    | ロック発生時は監査ログへ記録する      |

## WebSocket編集イベント制限

| 項目     | 内容                               |
|--------|----------------------------------|
| 目的     | 大量送信によるサーバー負荷を防止                 |
| 制限方式   | 接続ごとにToken Bucket方式で送信可能イベント数を制限 |
| バケット容量 | 接続ごとに最大30トークン                    |
| トークン補充 | 1秒あたり10トークンを補充する                 |
| 超過時    | リクエストを破棄                         |

## 通信セキュリティ

| 項目          | 内容                             |
|-------------|--------------------------------|
| API通信       | HTTPS必須                        |
| WebSocket通信 | WSSを必須、暗号化されていないWebSocket通信は禁止 |
| Cookie属性    | Secure、HttpOnly、SameSiteを設定    |
| CORS        | フロントエンドのドメインのみ許可               |

## リフレッシュトークン

| 項目      | 内容                                           |
|---------|----------------------------------------------|
| 保存場所    | HttpOnly Cookie                              |
| DB保存    | ハッシュ化した値のみ保存し、平文は保存しない                       |
| 有効期限    | AccessTokenより長い期間 14日                        |
| ローテーション | リフレッシュ時に新しいRefreshTokenを発行し、旧トークンを無効化する      |
| 再利用検知   | 無効化済みトークンの利用を検知した場合はセッションを強制ログアウトし、監査ログへ記録する |
| CSRF対策  | Cookieを利用するAPIはDouble Submit Cookie方式を採用する   |

## ログ要件

### API処理ログ

| 項目   | 内容                                                          |
|------|-------------------------------------------------------------|
| ログ項目 | request_id、method、path、status、latency_ms、user_id、error_code |

### 監査ログ

| 項目   | 内容                                                                         |
|------|----------------------------------------------------------------------------|
| 記録対象 | ログイン、ログアウト、RefreshToken再利用検知、ログイン試行制限発動、権限変更、ワークスペース作成・削除、ドキュメント削除、バッチ処理実行 |

### Rate Limit・WebSocketログ

| 項目   | 内容                                   |
|------|--------------------------------------|
| 記録対象 | レート制限超過、WebSocket強制切断、同時接続数超過による接続拒否 |

## XSS対策

| 項目 | 内容                                                                                   |
|----|--------------------------------------------------------------------------------------|
| 対象 | ドキュメント本文、タイトルなどユーザーが入力した内容                                                           |
| 対策 | 表示時にHTMLから安全な内容だけを表示、scriptタグ、iframeタグ、イベント属性（onClick等）、javascript: URLなどの危険な要素を除去する |

## インフラ・デプロイ要件

| 項目              | 内容                                                         |
|-----------------|------------------------------------------------------------|
| フロントエンド         | React、TypeScript、Vite、Tailwind CSSで実装し、Dockerコンテナとしてデプロイする |
| バックエンド          | Go（Echo）によるREST API・WebSocketサーバーをDockerコンテナ               |
| DB              | PostgreSQL（GORMを利用）                                        |
| Redis（Pub/Sub用） | WebSocketのPub/Subおよび編集者の管理に利用する                            |
| Redis（自動保存用）    | 編集中心ドキュメントのワーキングキャッシュとして利用し、60秒ごとにPostgreSQLへ永続化する         |
| コンテナ管理          | Docker Composeでフロントエンド、バックエンド、PostgreSQL、Redisを一括起動する      |
| CI/CD           | GitHub Actionsによるビルド、テスト、デプロイを自動化する                        |